

Рекурсивные функции

Рекурсивная функция - функция, которая вызывает саму себя

Базис рекурсии - ветка, в которой рекурсии не будет

Базисов может быть несколько

```
void f(int n) {  
    // Базис рекурсии  
    if (n < 0) {  
        return;  
    }  
    ...  
    // Рекурсивный вызов  
    f(n - 1);  
}
```

Шаг рекурсии - вызов функцией самой себя при изменении параметров

Виды рекурсии

- **Прямая**

- **Головная** (не хвостовая)

Рекурсивный вызов стоит не в конце

- **Концевая** (хвостовая)

Рекурсивный вызов выполняется последним
(даже при рекурсивном вызове в конце функции)

рекурсия не хвостовая, если с результатом этого вызова выполняются еще какие-то действия)

- **Косвенная**

```
int frac(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return n * fac(n - 1); // не хвостовая  
}
```

При хвостовой рекурсии, ее можно преобразовать в линейный (циклический) код

Косвенная рекурсия

```
void foo(){  
    ...  
    bar();  
}  
  
void bar() {  
    ...  
    foo()  
}  
  
int main() {  
    foo();  
}
```

Алгоритмы на основе рекурсии

Линейный поиск

Идея: Берем последний элемент массива и смотрим, является ли он искомым. Если не является, то рассматриваем следующий элемент с конца

```
int linear_search(int key, int* a, int n) {  
    // Если дошли до начала и ничего не нашли  
    if (n == 0) {  
        return -1;  
    }  
    // Если последний элемент искомый, то возвращаем  
его индекс  
    if (key == a[n-1]) {  
        return n - 1;  
    }  
    // Перемещаемся к следующему элементу  
    return linear_search(key, a, n - 1);  
}
```

Бинарный поиск

Идея: Отсортированный массив делим пополам и смотрим на элемент посередине. В зависимости от него определяем, с какой половиной будем работать дальше, пока не найдем нужный элемент

```
int binary_search(int key, int* a, int left, int  
right) {  
    // Если массив кончился и ничего не нашлось  
    if (left > right) return -1;  
  
    // Не оптимизируем выражение, чтобы результат  
точно влез в ячейку памяти  
    int middle = left + (right - left) / 2;  
  
    // Если вдруг элемент посередине искомый
```

```
    if (key == a[middle]) {
        return middle;
    }

    // В зависимости от элемента посередине смотрим с
    чем будем работать дальше
    if (key < a[middle]) {
        return binary_search(key, a, left, middle -
1);
    } else {
        return binary_search(key, a, middle + 1,
right);
    }
}
```

(Дополнительно)

`getch` - функция, возвращающая нажатую клавишу (`get char`)

| `getch` можно использовать для остановки программы

```
if (getch() == 123) {...}
```